Junnior Marcellino Polla

11231034

Sistem Paralel dan terdistribusi B

UTS

Link github:
https://github.com/junN0ir/Pub-sub-log-Aggregator

Link Youtube:
https://youtu.be/I2hHQgH165I

Ringkasan Sistem

Sistem ini merupakan Pub-Sub Log Aggregator yang dibangun menggunakan Python dengan framework FastAPI dan dijalankan di dalam Docker. Arsitektur sistem terdiri dari dua komponen utama yang berjalan sebagai container terpisah melalui Docker Compose:

- Publisher Service, bertanggung jawab mengirimkan event atau log ke Aggregator melalui endpoint POST /publish. Publisher mampu mengirim event dalam bentuk batch, termasuk event duplikat, untuk mensimulasikan kondisi nyata pengiriman ulang akibat retry atau kegagalan jaringan.
- Aggregator Service, menerima event dari publisher, melakukan validasi skema menggunakan Pydantic, memeriksa duplikasi berdasarkan kombinasi (topic, event_id), menyimpan event unik ke SQLite sebagai persistent deduplication store, dan menolak event duplikat tanpa menimbulkan efek samping. Service ini juga menyediakan endpoint GET /events dan GET /stats untuk keperluan monitoring dan observabilitas sistem.

Kedua service berjalan dalam jaringan internal Docker Compose tanpa ketergantungan layanan eksternal publik. Deduplication store disimpan di Docker volume lokal sehingga data tetap persisten meskipun container di-restart.

Keputusan Desain

Desain sistem didasarkan pada empat macam aspek yaitu idempotency, deduplication store, ordering, dan retry mechanism.

Idempotency diimplementasikan dengan menjadikan kombinasi (topic, event_id) sebagai primary key di tabel SQLite. Sebelum event diproses, consumer memeriksa apakah kombinasi tersebut sudah tercatat. Jika sudah ada, event diabaikan dan counter duplicate_dropped bertambah tanpa menimbulkan efek tambahan. Pendekatan ini memastikan sistem tetap konsisten meskipun publisher mengirim event yang sama berkali-kali sesuai semantik at-least-once delivery.

Deduplication store menggunakan SQLite embedded sebagai basis data lokal yang menyimpan pasangan (topic, event_id) beserta metadata event. SQLite dipilih karena tidak memerlukan service eksternal, mendukung atomicity melalui mekanisme INSERT OR IGNORE, dan bersifat durable. Data disimpan di path /app/data/dedup.db yang di-mount sebagai Docker named volume sehingga informasi deduplication tetap ada setelah container dimatikan maupun di-restart.

Ordering tidak diterapkan secara total antar topic karena overhead koordinasi global tidak sebanding dengan manfaatnya untuk sistem log aggregator. Urutan antar event dari sumber berbeda tidak memengaruhi kebenaran

data. Sebagai gantinya sistem mengandalkan field timestamp pada setiap event untuk menjaga causal ordering yang cukup dalam satu topic, sesuai dengan prinsip partial ordering yang dibahas Van Steen dan Tanenbaum (2023, hlm. 260–264).

Retry mechanism diterapkan di sisi publisher. Jika pengiriman event gagal, publisher dapat mengirim ulang event yang sama tanpa risiko duplikasi di sisi aggregator karena consumer bersifat idempotent. Kombinasi at-least-once delivery di publisher dan idempotent consumer di aggregator menghasilkan efek setara exactly-once di tingkat aplikasi.

Analisis Performa dan Metrik

Uji performa dilakukan dengan mengirimkan 5.000 event di mana 20% di antaranya merupakan duplikat. Pengujian menggunakan script PowerShell yang mengirim seluruh event dalam satu batch request ke POST /publish. Sistem berhasil memproses seluruh event dan tetap responsif selama pengujian berlangsung.

Tiga metrik utama yang digunakan dalam evaluasi:

1. Throughput , jumlah event unik yang berhasil diproses dan disimpan per satuan waktu. Tren sehat ditunjukkan dengan nilai unique_processed yang mendekati 80% dari total received.
2. Latency , waktu dari event diterima endpoint hingga ditulis ke dedup store. asyncio.Queue memastikan endpoint tidak terblokir selama consumer memproses event di background.
3. Duplicate Rate , rasio duplicate_dropped terhadap received. Hasil pengujian menunjukkan nilai sekitar 20% sesuai proporsi duplikat yang dikirimkan, membuktikan mekanisme deduplication bekerja sesuai ekspektasi.

SQLite menjadi bottleneck pada aspek I/O untuk skala sangat besar, namun trade-off ini diterima untuk menjamin durability dan kemampuan recovery setelah kegagalan sistem. Sistem tetap responsif karena menggunakan antrian asinkron berbasis asyncio.Queue dengan kapasitas 50.000 slot dan pemrosesan non-blocking I/O.

Soal Teori

1. Jelaskan karakteristik utama sistem terdistribusi dan trade-off yang umum pada desain Pub-Sub log aggregator. T1 (Bab 1)
2. Bandingkan arsitektur client-server vs publish-subscribe untuk aggregator. Kapan memilih Pub-Sub? Berikan alasan teknis. T2 (Bab 2)
3. Uraikan at-least-once vs exactly-once delivery semantics. Mengapa idempotent consumer krusial di presence of retries? T3 (Bab 3)
4. Rancang skema penamaan untuk topic dan event_id (unik, collision-resistant). Jelaskan dampaknya terhadap dedup. T4 (Bab 4)
5. Bahas ordering: kapan total ordering tidak diperlukan? Usulkan pendekatan praktis (mis. event timestamp + monotonic counter) dan batasannya.
6. Identifikasi failure modes (duplikasi, out-of-order, crash). Jelaskan strategi mitigasi (retry, backoff, durable dedup store). T5 (Bab 5)
7. Definisikan eventual consistency pada aggregator; jelaskan bagaimana idempotency + dedup membantu mencapai konsistensi. T6 (Bab 6)
8. Rumuskan metrik evaluasi sistem (throughput, latency, duplicate rate) dan kaitkan ke keputusan desain. T7 (Bab 7)

Jawaban Teori

1. Sistem terdistribusi dicirikan oleh empat tujuan desain utama: resource sharing, distribution transparency, openness, serta dependability dan scalability (Van Steen & Tanenbaum, 2023, hlm. 10–14). Namun, peningkatan transparency selalu berbayar: semakin tinggi upaya menyembunyikan distribusi, semakin besar overhead koordinasi yang menurunkan performa dan skalabilitas (hlm. 14–15).

   Untuk desain Pub-Sub log aggregator, tiga trade-off utama yang muncul adalah sebagai berikut. Pertama, decoupling temporal dan referensial meningkatkan elastisitas dan availability, tetapi memindahkan beban matching dan buffering ke broker sehingga menambah latensi. Kedua, at-least-once via retry meningkatkan reliabilitas tetapi memicu duplikasi, sedangkan exactly-once sulit dan mahal dicapai sehingga solusi realistisnya adalah idempotent consumer dikombinasikan dengan durable dedup store. Ketiga, replikasi memfasilitasi penskalaan tetapi menantang konsistensi, sehingga arsitektur perlu toleran out-of-order dan memilih ordering yang lebih lemah seperti FIFO atau causal jika memadai (Van Steen & Tanenbaum, 2023, hlm. 68–73).

2. Perbedaan mendasar keduanya terletak pada dimensi coupling antar proses sebagaimana diklasifikasikan dalam Figure 2.9 (Van Steen & Tanenbaum, 2023, hlm. 69). Client-server bersifat temporally dan referentially coupled: klien dan server harus aktif bersamaan dan saling mengenal secara eksplisit,

menjadikannya cocok untuk interaksi sinkron, low latency, dan jalur kontrol sempit. Publish-subscribe sebaliknya bersifat temporally dan referentially decoupled: komunikasi berbasis deskripsi topik bukan identitas endpoint, memungkinkan fan-out besar, dynamic membership, dan persistent asynchronous communication melalui broker (hlm. 70–73).
Pilih Pub-Sub ketika tiga kondisi berikut terpenuhi: produsen dan konsumen tidak perlu aktif bersamaan, penyebaran ke banyak konsumen diperlukan, dan beban burst ingin dialihkan ke broker. Pilih client-server jika interaksi bersifat sessionful, deterministik, dan low latency adalah prioritas utama (Van Steen & Tanenbaum, 2023, hlm. 68–73)

3. Terdapat tiga semantik pengiriman pesan yang dibahas dalam konteks message-oriented persistent communication (Van Steen & Tanenbaum, 2023, hlm. 220–226). At-most-once: pesan dikirim paling banyak sekali, berisiko kehilangan pesan. At-least-once: pesan dijamin terkirim minimal sekali melalui retransmit, tetapi membuka peluang duplikasi. Exactly-once: ideal tanpa kehilangan maupun duplikasi, namun memerlukan koordinasi end-to-end yang mahal dan rentan blocking saat terjadi kegagalan.
Karena retry adalah respons wajar terhadap ketidakpastian jaringan, idempotent consumer menjadi krusial: operasi yang sama dapat diulang tanpa mengubah hasil akhir. Secara praktis, konsumen menyimpan kunci dedup (topic, event_id) untuk mendeteksi dan mengabaikan pengiriman ulang, sehingga meskipun infrastruktur hanya menawarkan at-least-once, efek di tingkat aplikasi setara dengan exactly-once (Van Steen & Tanenbaum, 2023, hlm. 220–226).

4. Menurut Van Steen dan Tanenbaum (2023, hlm. 326–329), penting membedakan tiga konsep: name (label untuk merujuk entitas), identifier (name yang unik, tidak dapat didaur ulang, dan selalu merujuk entitas yang sama), serta address (access point yang dapat berubah). Menggunakan address sebagai identifier adalah praktik buruk karena menghasilkan referensi yang tidak valid begitu access point berubah (hlm. 327–328).
Skema yang direkomendasikan adalah sebagai berikut. Untuk topic: gunakan structured naming hierarkis seperti logs.app.service.v1 sesuai prinsip hlm. 344–352, untuk memfasilitasi routing dan filtering tanpa mengkodekan lokasi. Untuk event_id: gunakan flat identifier seperti UUID v7, ULID, atau hash kriptografis dari (topic, producer_id, timestamp, nonce) sesuai prinsip hlm. 329–343, guna memperoleh keunikan dan collision resistance.
Dampak terhadap deduplication: kunci (topic, event_id) yang location-independent memungkinkan consumer melakukan set-membership check pada dedup store; insert pertama diterima sebagai unik, insert berikutnya dengan kunci identik ditolak, dan jika terjadi tabrakan ID, content hash berfungsi sebagai tie-breaker (Van Steen & Tanenbaum, 2023, hlm. 326–329).

5. Lamport menunjukkan bahwa yang terpenting bukan kesepakatan waktu absolut, melainkan kesepakatan urutan happens-before; logical clock sudah cukup untuk banyak kasus tanpa memerlukan jam fisik tersinkronisasi (Van Steen & Tanenbaum, 2023, hlm. 260–261). Total ordering hanya diperlukan ketika urutan operasi memengaruhi hasil akhir secara kritis, seperti pada replicated database yang membutuhkan totally ordered multicast (hlm. 263–264). Untuk log aggregator tanpa dependensi kuat antar-event, causal atau FIFO ordering per-topic sudah memadai.
Pendekatan praktis yang dapat diterapkan adalah menggunakan tuple (event_timestamp, monotonic counter per-producer) sebagai tiebreaker, kemudian melakukan stable sort berdasarkan (topic, timestamp, counter) dengan toleransi skew dalam window tertentu. Adapun batasannya: Lamport clocks tidak membedakan peristiwa concurrent (hlm. 261–262); vector clocks memberikan informasi kausal lebih kaya namun dengan overhead metadata lebih besar (hlm. 266–271); dan buffering untuk reordering menambah latency ekor (Van Steen & Tanenbaum, 2023, hlm. 260–266).

6. Sebagaimana dibahas dalam failure models (Van Steen & Tanenbaum, 2023, hlm. 466–470), tiga mode kegagalan utama yang relevan pada log aggregator adalah: duplikasi pesan akibat retry, out-of-order delivery karena variabilitas latensi jaringan, dan crash atau restart pada komponen sistem. Desain harus mengakui bahwa partial failure adalah kondisi normal dalam sistem terdistribusi, bukan pengecualian (hlm. 462–463).
   Strategi mitigasi yang perlu diterapkan mencakup beberapa lapis. Untuk duplikasi: gunakan durable dedup store persisten menyimpan (topic, event_id) agar idempotensi tetap berlaku setelah restart, dikombinasikan dengan retry menggunakan exponential backoff dan jitter untuk mencegah thundering herd. Untuk out-of-order: sediakan reorder buffer kecil berbasis sequence number per-producer. Untuk crash: gunakan persistent queue dan durable messages agar tidak kehilangan data. Untuk pesan bermasalah: pisahkan ke dead-letter queue agar tidak menyumbat aliran utama. Terakhir, bangun observability melalui metrik duplicate rate, panjang antrean, dan p99 latency sesuai prinsip failure masking by redundancy pada hlm. 470–471 (Van Steen & Tanenbaum, 2023, hlm. 462–470).

7. Eventual consistency menyatakan bahwa selama tidak ada write-write conflict, seluruh replika akan berkonvergensi pada keadaan yang sama setelah propagasi update selesai; model ini dipilih karena hemat koordinasi dan sering cukup untuk banyak skenario seperti DNS dan Web caches (Van Steen & Tanenbaum, 2023, hlm. 406–407).

   Dalam log aggregator, mekanisme pencapaiannya adalah sebagai berikut. Idempotent consumer memastikan retry tidak mengubah hasil, sehingga jumlah pengiriman ulang tidak memengaruhi keadaan akhir. Deduplication store mencegah double-apply: setiap pasangan unik (topic, event_id) hanya diproses sekali, duplikat ditolak. Ketika konflik tak terhindarkan, resolusi sederhana seperti last-writer-wins berbasis waktu yang disepakati dapat diterapkan sebagaimana dibahas pada hlm. 408–409. Hasilnya, kombinasi at-least-once delivery + idempotency + dedup menghasilkan eventual consistency pragmatis tanpa biaya global total order (Van Steen & Tanenbaum, 2023, hlm. 406–409).

8. Evaluasi sistem terdistribusi memerlukan metrik yang terkait langsung dengan keputusan desain (Van Steen & Tanenbaum, 2023, hlm. 24–31). Metrik utama yang perlu dipantau adalah sebagai berikut. Throughput (event/detik): mengukur kapasitas paralel ingest; tren sehat naik mendekati linear seiring penambahan worker hingga batas I/O. Latency (p95/p99): menilai delay dari publish hingga durable write; peningkatan p99 adalah sinyal tuning batch size atau paralelisme. Duplicate rate (duplicate_dropped / received): mengukur efektivitas deduplication; boleh berfluktuasi saat jaringan tidak stabil asalkan hasil akhir konsisten. Queue lag (selisih event timestamp ke commit): indikator backpressure. Error rate: memvalidasi fault handling secara keseluruhan.
   Keterkaitan dengan keputusan desain adalah sebagai berikut. Kunci idempoten (topic, event_id) + content hash menekan duplicate rate. Durable dedup store menjamin exactly-once effect di sisi konsumen. Ordering berbasis timestamp + counter tanpa global total order menjaga throughput tinggi sesuai prinsip hlm. 260–264. Retry dengan backoff pada message-queuing menjaga error rate rendah sesuai pembahasan hlm. 220–226

9. Evaluasi sistem terdistribusi harus mencakup seperangkat metrik yang saling terkait untuk memberikan gambaran menyeluruh tentang kesehatan sistem. Throughput yang diukur dalam event per detik mencerminkan efisiensi jalur ingest mencakup validasi skema, enqueue, dan upsert ke dedup store, sekaligus menunjukkan kapasitas paralel sistem; menurut pembahasan scalability pada Van Steen dan Tanenbaum (2023, hlm. 24–31), tren throughput yang sehat naik mendekati linear seiring penambahan

worker hingga batas I/O, dan jika antrean mulai memanjang itu merupakan sinyal perlunya tuning ukuran batch, paralelisme, atau frekuensi komit database. Latency ujung-ke-ujung, khususnya p95 dan p99, menilai processing delay dari saat event dipublikasikan hingga durable write selesai, dan duplicate rate yang dihitung sebagai rasio duplicate_dropped terhadap received mengukur efektivitas deduplication dalam menangani retry; nilai duplicate rate ini wajar berfluktuasi saat jaringan tidak stabil namun hasil akhir harus tetap konsisten karena duplikat berhasil ditolak sebagaimana dijamin oleh mekanisme idempoten. Metrik tambahan seperti error rate, queue lag yaitu selisih antara event timestamp dan waktu commit, serta waktu konvergensi pada model eventual consistency sebagaimana dibahas pada hlm. 406–409 turut melengkapi gambaran sistem. Seluruh metrik ini saling berkaitan dengan keputusan desain: kunci idempoten (topic, event_id) ditambah content hash untuk conflict detection, durable dedup store untuk exactly-once effect di sisi konsumen, ordering praktis berbasis timestamp dan counter tanpa global total order yang sesuai prinsip pada hlm. 260–264, serta retry dengan backoff pada message-queuing yang dibahas pada hlm. 220–226, semuanya merupakan kompromi yang diambil untuk menyeimbangkan performa, reliabilitas, dan konsistensi pada sistem terdistribusi yang skalabel (Van Steen & Tanenbaum, 2023, hlm. 24–31).

Implementasi

1. Model Event dan API

Sistem dibangun menggunakan FastAPI dengan asyncio sebagai fondasi utama. Model event didefinisikan menggunakan Pydantic untuk validasi otomatis setiap field yang masuk.

Struktur event JSON minimal yang diterima sistem:

```
{
  "topic": "string",
  "event_id": "string-unik",
  "timestamp": "ISO8601",
  "source": "string",
  "payload": { ... }
}
```

```python
class Event(BaseModel):
    topic: str
    event_id: str
    timestamp: str
    source: str
    payload: dict[str, Any] = {}

    @field_validator("topic")
    @classmethod
    def topic_not_empty(cls, v: str) -> str:
        if not v or not v.strip():
            raise ValueError("topic tidak boleh kosong")
        return v.strip()

    @field_validator("event_id")
    @classmethod
    def event_id_not_empty(cls, v: str) -> str:
        if not v or not v.strip():
            raise ValueError("event_id tidak boleh kosong")
        return v.strip()

    @field_validator("timestamp")
    @classmethod
    def timestamp_valid(cls, v: str) -> str:
        try:
            datetime.fromisoformat(v.replace("Z", "+00:00"))
        except ValueError:
            raise ValueError("timestamp harus format ISO8601 (contoh: 2024-01-01T00:00:00Z)")
        return v

    @field_validator("source")
    @classmethod
    def source_not_empty(cls, v: str) -> str:
        if not v or not v.strip():
            raise ValueError("source tidak boleh kosong")
        return v.strip()
```

Validasi dilakukan otomatis oleh Pydantic , topic dan event_id tidak boleh kosong, timestamp harus format ISO8601, dan source wajib diisi. Jika salah satu field tidak valid, sistem mengembalikan HTTP 422 secara otomatis.

Endpoint yang tersedia:

- POST /publish — menerima batch atau single event, memasukkan ke asyncio.Queue
- GET /events?topic=... — mengembalikan daftar event unik berdasarkan topic
- GET /stats — menampilkan received, unique_processed, duplicate_dropped, topics, uptime
- GET /health — health check untuk Docker

```python
@app.post("/publish", status_code=202)
async def publish(request: PublishRequest):
    """
    Terima batch event dari publisher.
    Event dimasukkan ke internal asyncio.Queue untuk diproses consumer.
    """
    if not request.events:
        raise HTTPException(status_code=400, detail="Tidak ada event dalam request.")

    enqueued = await enqueue_events(request.events)

    return {
        "status": "accepted",
        "total_submitted": len(request.events),
        "enqueued": enqueued,
        "message": f"{enqueued} event diterima untuk diproses."
    }
```

```python
@app.get("/events")
async def get_events(topic: str = Query(..., description="Nama topic yang ingin dilihat")):
    """
    Kembalikan daftar event unik yang sudah diproses untuk topic tertentu.
    """
    events = dedup_store.get_events_by_topic(topic)
    return {
        "topic": topic,
        "count": len(events),
        "events": events,
    }
```

```python
@app.get("/stats", response_model=StatsResponse)
async def get_stats():
    """
    Tampilkan statistik sistem:
    received, unique_processed, duplicate_dropped, topics, uptime.
    """
    stats = consumer.get_stats()
    topics = dedup_store.get_all_topics()
    uptime = time.time() - START_TIME

    return StatsResponse(
        received=stats["received"],
        unique_processed=stats["unique_processed"],
        duplicate_dropped=stats["duplicate_dropped"],
        topics=topics,
        uptime_seconds=round(uptime, 2),
    )
```

```python
@app.get("/health")
async def health():
    return {"status": "ok", "timestamp": datetime.now(timezone.utc).isoformat()}
```

2. Idempotency dan Deduplication

Sistem mengimplementasikan idempotent consumer menggunakan SQLite sebagai persistent deduplication store. Primary key tabel processed_events adalah kombinasi (topic, event_id) sehingga kombinasi yang sama tidak bisa tersimpan dua kali

Alur kerja deduplication:

1. Event masuk dari POST /publish ke asyncio.Queue
2. Consumer membaca event dari queue satu per satu
3. Sebelum diproses, consumer memanggil is_duplicate(topic, event_id) ke DedupStore
4. Jika duplikat maka event di-drop, counter duplicate_dropped bertambah, log WARNING ditulis
5. Jika baru maka event disimpan via mark_processed() menggunakan INSERT OR IGNORE

```python
def is_duplicate(self, topic: str, event_id: str) -> bool:
    """Return True jika event sudah pernah diproses."""
    with self._lock:
        conn = self._get_conn()
        try:
            cur = conn.execute(
                "SELECT 1 FROM processed_events WHERE topic=? AND event_id=?",
                (topic, event_id)
            )
            return cur.fetchone() is not None
        finally:
            conn.close()

def mark_processed(self, topic: str, event_id: str, source: str,
                   timestamp: str, payload: str) -> bool:
    """
    Tandai event sebagai processed.
    Return True jika berhasil insert (baru), False jika duplikat (sudah ada).
    """
    import json
    processed_at = datetime.now(timezone.utc).isoformat()
    with self._lock:
        conn = self._get_conn()
        try:
            conn.execute(
                """INSERT OR IGNORE INTO processed_events
                   (topic, event_id, source, timestamp, payload, processed_at)
                   VALUES (?,?,?,?,?,?)""",
                (topic, event_id, source, timestamp,
                 json.dumps(payload) if isinstance(payload, dict) else payload,
                 processed_at)
            )
            conn.commit()
            # Cek apakah insert berhasil (rowcount > 0 berarti baru)
            cur = conn.execute(
                "SELECT processed_at FROM processed_events WHERE topic=? AND event_id=?",
                (topic, event_id)
            )
            row = cur.fetchone()
            return row is not None and row[0] == processed_at
        finally:
            conn.close()
```

Penggunaan INSERT OR IGNORE menjamin atomicity , bahkan jika dua worker mencoba insert event yang sama secara bersamaan (race condition), hanya satu yang berhasil.

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/publish" -Method Post `
>>   -ContentType "application/json" -Body '{
>>     "events": [{
>>       "topic": "demo.logs",
>>       "event_id": "evt-001", …
>>       "source": "demo_client",
>>       "payload": { "msg": "event pertama" }
>>     }]
>>   }'

status   total_submitted enqueued message
------   --------------- -------- -------
accepted               1        1 1 event diterima untuk diproses.
```

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/publish" -Method Post `
>>   -ContentType "application/json" -Body '{
>>     "events": [{
>>       "topic": "demo.logs",
>>       "event_id": "evt-001",
>>       "timestamp": "2025-01-01T10:00:00Z",
>>       "source": "demo_client",
>>       "payload": { "msg": "duplikat, harus di-drop" }
>>     }]
>>   }'

status   total_submitted enqueued message
------   --------------- -------- -------
accepted               1        1 1 event diterima untuk diproses.
```

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/events?topic=demo.logs"

topic     count events
-----     ----- ------
demo.logs     1 {@{topic=demo.logs; event_id=evt-001; source=demo_client; timestamp=2025-01-01T10:00:00Z; payload=; processed_at=2026-04-23T20:01:58.880243+...
```

3. Reliability dan Ordering

At-least-once Delivery

Sistem dirancang untuk menangani skenario at-least-once delivery, yaitu kondisi di mana publisher mengirim ulang event yang sama akibat timeout, retry, atau network error. Publisher (publisher.py) secara eksplisit mengirim 25% event duplikat untuk mensimulasikan kondisi ini.

Toleransi Crash (Persistent Dedup Store)

SQLite database disimpan di path /app/data/dedup.db di dalam container. Path ini di-mount sebagai Docker volume (-v aggregator-data:/app/data) sehingga data tetap ada meskipun container di-stop atau di-restart. Saat startup, sistem memuat ulang jumlah event yang sudah diproses dari database sehingga counter unique_processed tidak reset ke nol.

Ordering

Total ordering tidak diperlukan dalam konteks log aggregator ini. Tujuan utama sistem adalah mengumpulkan dan menyimpan log unik dari berbagai sumber — urutan antar event dari source berbeda tidak memengaruhi kebenaran data. Partial ordering via field timestamp sudah cukup untuk keperluan audit dan penelusuran log. Pendekatan ini juga menghindari overhead koordinasi global yang mahal.

```
● (venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> docker ps
  CONTAINER ID   IMAGE            COMMAND                  CREATED         STATUS                   PORTS                                          NAMES
  dec1e211f683   uts-aggregator   "python -m uvicorn s…"   8 minutes ago   Up 8 minutes (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp    aggregator-demo
● (venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> docker restart aggregator-demo
  aggregator-demo
● (venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Write-Host "Menunggu 5 detik setelah restart..."
  Menunggu 5 detik setelah restart...
● (venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Start-Sleep -Seconds 5
○ (venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator>
```

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/publish" -Method Post `
>>   -ContentType "application/json" -Body '{
>>     "events": [{
>>       "topic": "demo.logs",
>>       "event_id": "evt-001",
>>       "timestamp": "2025-01-01T10:00:00Z",
>>       "source": "demo_client",
>>       "payload": { "msg": "ulang setelah restart" }
>>     }]
>>   }'

status   total_submitted enqueued message
------   --------------- -------- -------
accepted               1        1 1 event diterima untuk diproses.


(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/stats"


received          : 1
unique_processed  : 5004
duplicate_dropped : 1
topics            : {demo.logs, logs.app, logs.auth, logs.db...}
uptime_seconds    : 184.04



(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator>
```

4. Performa Minimum

Sistem diuji dengan mengirim 5.000 event sekaligus di mana 20% di antaranya adalah duplikat (1.000 event duplikat, 4.000 event unik). Pengujian dilakukan menggunakan script PowerShell yang membuat array event dan mengirimnya dalam satu request batch ke POST /publish. Hasil yang diharapkan setelah stress test ialah agar sistem tetap responsif, endpoint /health tetap menjawab selama pengiriman

asyncio.Queue dengan maxsize=50.000 memastikan tidak ada event yang dibuang selama load tinggi. Consumer memproses event secara asinkron di background tanpa memblokir endpoint HTTP.

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> for ($i = 1; $i -le 5000; $i++) {
>>     $id = if ($i % 5 -eq 0) { "evt-$($i - 1)" } else { "evt-$i" }
>>     $events += @{
>>         topic     = "stress.logs"
>>         event_id  = $id
>>         timestamp = (Get-Date).ToString("yyyy-MM-ddTHH:mm:ssZ")
>>         source    = "stress_tester"
>>         payload   = @{ msg = "Event $i" }
>>     }
>> }
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> $jsonBody = @{ events = $events } | ConvertTo-Json -Depth 5
>> Invoke-RestMethod -Uri "http://localhost:8080/publish" -Method Post `
>>   -ContentType "application/json" -Body $jsonBody

status   total_submitted enqueued message
------   --------------- -------- -------
accepted            5000     5000 5000 event diterima untuk diproses.


(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator>
```

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Start-Sleep -Seconds 3
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> Invoke-RestMethod -Uri "http://localhost:8080/stats"


received          : 5001
unique_processed  : 9004
duplicate_dropped : 1001
topics            : {demo.logs, logs.app, logs.auth, logs.db...}
uptime_seconds    : 637.81



(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator>
```

5. Docker

Sistem dikemas dalam Docker image menggunakan python:3.11-slim sebagai base image untuk ukuran image yang minimal. Beberapa praktik keamanan diterapkan:

- **Non-root user** ,dibuat user appuser agar container tidak berjalan sebagai root
- **Dependency caching**, requirements.txt di-copy dan di-install sebelum source code, sehingga layer pip install hanya dieksekusi ulang jika requirements berubah
- **Health check**, Dockerfile dan docker-compose.yml keduanya mendefinisikan health check ke /health agar container status termonitor
- **Volume mount**, direktori /app/data di-mount sebagai named volume untuk persistensi SQLite

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> docker build -t uts-aggregator .
[+] Building 46.4s (12/12) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 853B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/7] FROM docker.io/library/python:3.11-slim@sha256:92c262cbb2e99cdc16218338d74fbe518055c13d224d942708f70f8042ff6d18
 => => resolve docker.io/library/python:3.11-slim@sha256:92c262cbb2e99cdc16218338d74fbe518055c13d224d942708f70f8042ff6d18
 => => sha256:3531af2bc2a9c8883754652783cf96207d53189db279c9637b7157d034de7ecd 29.78MB / 29.78MB
 => => sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d765d7c799d46035ceb563 249B / 249B
 => => sha256:f3ba2250c52436e41e9f2f3830af92de466fcacc2d848264e1f9fdc76975d803 14.37MB / 14.37MB
 => => sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e13f94fa2f12ae4b221aae996b9 1.29MB / 1.29MB
 => => extracting sha256:3531af2bc2a9c8883754652783cf96207d53189db279c9637b7157d034de7ecd
 => => extracting sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e13f94fa2f12ae4b221aae996b9
 => => extracting sha256:f3ba2250c52436e41e9f2f3830af92de466fcacc2d848264e1f9fdc76975d803
 => => extracting sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d765d7c799d46035ceb563
 => [internal] load build context
 => => transferring context: 109.90kB
 => [2/7] WORKDIR /app
 => [3/7] RUN adduser --disabled-password --gecos '' appuser     && mkdir -p /app/data     && chown -R appuser:appuser /app
 => [4/7] COPY --chown=appuser:appuser requirements.txt ./
 => [5/7] RUN pip install --no-cache-dir -r requirements.txt
 => [6/7] COPY --chown=appuser:appuser src/ ./src/
 => [7/7] COPY --chown=appuser:appuser tests/ ./tests/
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:30739180106862afa15d4c07bc724ecf150e37c78fa1c5359e5d0740a0d8de59
 => => exporting config sha256:ba63d2313b579ca18f07bff8281ab859d47a21281cbdc22d6afdfea92a2e972b
 => => exporting attestation manifest sha256:a5712bcfb0d40841be0de6851ffacd930fee5e32e448b0b1c9de9fac627b8b32
 => => exporting manifest list sha256:7abebef2f648d17a8760c20b136178b6e5e319023a536a99f08067583152a433
 => => naming to docker.io/library/uts-aggregator:latest
 => => unpacking to docker.io/library/uts-aggregator:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/gk9yz0f75bml776j4ndbqyw1f
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator>
```

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> docker run -d --name aggregator-demo -p 8080:8080 -v aggregator-data:/app/data uts-aggregator
8678d5af7f74d64a551b8ea3533e3e831d2ecd191e120741fb0531cece637b5f
```

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> docker ps
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS                    PORTS                                          NAMES
8678d5af7f74   uts-aggregator   "python -m uvicorn s…"   24 seconds ago   Up 24 seconds (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp    aggregator-dem
o
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator>
```

6. Docker Compose

```
(venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> docker-compose up --build
#21 exporting config sha256:529814baf8c46f9a1a7728c8ab0c473e6a60c58037bfc396747329a752539f52 0.0s done
#21 exporting attestation manifest sha256:85e09d95ab70482c046353f84d07fb5a186fd7eec19d159ca958e3ee8e56547b 0.0s done
#21 exporting manifest list sha256:75cfafdfc85748bdaa9cdfd78006837e203975ccc685d99f51a14f7a84910073 0.0s done
#21 naming to docker.io/library/uts-aggregator-publisher:latest done
#21 unpacking to docker.io/library/uts-aggregator-publisher:latest
#21 unpacking to docker.io/library/uts-aggregator-publisher:latest 0.3s done
#21 DONE 1.6s

#22 [publisher] resolving provenance for metadata file
#22 DONE 0.0s
[+] up 4/4
 ✓ Image uts-aggregator-publisher  Built                                    16.9s
 ✓ Image uts-aggregator-aggregator Built                                    16.9s
 ✓ Container uts-aggregator        Created                                   0.1s
 ✓ Container uts-publisher         Created                                   0.1s
Attaching to uts-aggregator, uts-publisher
Container uts-aggregator Waiting
uts-aggregator  | INFO:     Started server process [1]
uts-aggregator  | INFO:     Waiting for application startup.
uts-aggregator  | 2026-04-23T21:07:28 [INFO] src.main: === Aggregator starting up ===
uts-aggregator  | 2026-04-23T21:07:28 [INFO] src.dedup_store: DedupStore initialized at /app/data/dedup.db
uts-aggregator  | 2026-04-23T21:07:28 [INFO] src.consumer: EventConsumer started.
uts-aggregator  | 2026-04-23T21:07:28 [INFO] src.main: Restored 0 previously processed events from dedup store.
uts-aggregator  | INFO:     Application startup complete.
uts-aggregator  | INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
uts-aggregator  | INFO:     127.0.0.1:53834 - "GET /health HTTP/1.1" 200 OK
Container uts-aggregator Healthy
uts-publisher   | ============================================================
uts-publisher   |   Pub-Sub Log Aggregator — Publisher Demo
uts-publisher   | ============================================================
uts-publisher   | Menunggu aggregator di http://aggregator:8080/health...
uts-aggregator  | INFO:     127.0.0.1:33722 - "GET /health HTTP/1.1" 200 OK
uts-aggregator  | INFO:     127.0.0.1:51724 - "GET /health HTTP/1.1" 200 OK
```

Docker Compose mendefinisikan dua service terpisah dalam satu jaringan internal:

- **Aggregator**, service utama FastAPI, port 8080, dengan volume dedup-data untuk persistensi SQLite. Service ini dilengkapi health check sehingga publisher tidak akan memulai sebelum aggregator benar-benar siap menerima request.

- **Publisher**, script Python yang mengirim 5.000+ event dengan 25% duplikat. Menggunakan depends_on dengan condition: service_healthy sehingga publisher otomatis menunggu aggregator healthy sebelum mulai mengirim.

Kedua service terhubung melalui jaringan internal aggregator-net yang dibuat otomatis oleh Compose. Tidak ada port yang diekspos ke luar selain port 8080 aggregator, dan tidak ada layanan eksternal publik yang digunakan

7. Unit Tests

Pengujian dilakukan menggunakan pytest dengan total 13 test yang dibagi ke dalam 5 file terpisah sesuai domain pengujiannya.

File	Jumlah Test	Yang Diuji
test_schema.py	4	Validasi model Event: timestamp invalid, topic kosong, source kosong
test_dedup.py	3	DedupStore: event baru bukan duplikat, deteksi duplikat, idempotent mark
test_persistence.py	2	Persistensi setelah simulasi restart, topic berbeda event_id sama
test_api.py	5	Endpoint /health, /publish, konsistensi /stats dan /events, batch rejected
test_stress.py	2	1000 event + 500 duplikat selesai < 10 detik, consumer idempotency

Setiap test menggunakan database SQLite sementara di /tmp/test_dedup.db yang dibersihkan sebelum dan sesudah setiap test melalui fixture autouse di conftest.py.

```
(.venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator> pytest tests/ -v
d:\belajar hebat\Sister\uts-pub-sub\.venv\Lib\site-packages\pytest_asyncio\plugin.py:208: PytestDeprecationWarning: The
 configuration option "asyncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asy
ncio will default the loop scope for asynchronous fixtures to function scope. Set the default fixture loop scope explic
itly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module",
 "package", "session"

  warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))
============================================= test session starts =============================================
platform win32 -- Python 3.11.9, pytest-8.3.3, pluggy-1.6.0 -- d:\belajar hebat\Sister\uts-pub-sub\.venv\Scripts\python
.exe
cachedir: .pytest_cache
rootdir: D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator
plugins: anyio-4.6.0, asyncio-0.24.0
asyncio: mode=Mode.STRICT, default_loop_scope=None
collected 16 items

tests/test_api.py::test_health_endpoint PASSED                                                          [  6%]
tests/test_api.py::test_publish_and_stats_consistent PASSED                                             [ 12%]
tests/test_api.py::test_publish_duplicate_reflected_in_stats PASSED                                     [ 18%]
tests/test_api.py::test_get_events_consistent_with_publish PASSED                                       [ 25%]
tests/test_api.py::test_publish_empty_batch_rejected PASSED                                             [ 31%]
tests/test_dedup.py::test_new_event_not_duplicate PASSED                                                [ 37%]
tests/test_dedup.py::test_detects_duplicate_after_mark PASSED                                           [ 43%]
tests/test_dedup.py::test_mark_processed_idempotent PASSED                                              [ 50%]
tests/test_persistence.py::test_persists_after_restart PASSED                                           [ 56%]
tests/test_persistence.py::test_different_topic_same_event_id PASSED                                    [ 62%]
tests/test_schema.py::test_event_schema_valid PASSED                                                    [ 68%]
tests/test_schema.py::test_event_schema_invalid_timestamp PASSED                                        [ 75%]
tests/test_schema.py::test_event_schema_empty_topic PASSED                                              [ 81%]
tests/test_schema.py::test_event_schema_empty_source PASSED                                             [ 87%]
tests/test_stress.py::test_stress_dedup_store PASSED                                                    [ 93%]
tests/test_stress.py::test_consumer_idempotency PASSED                                                  [100%]

============================================== 16 passed in 12.56s ==============================================
(.venv) PS D:\belajar hebat\Sister\uts-pub-sub\uts-aggregator>
```

Kesimpulan

Sistem Pub-Sub Log Aggregator berhasil diimplementasikan sesuai spesifikasi yang ditetapkan. Sistem mampu menerima event dari publisher, mendeteksi dan menolak duplikat berdasarkan kombinasi (topic, event_id), serta

mempertahankan state deduplication meskipun container di-restart berkat penggunaan SQLite yang di-mount sebagai Docker volume.

Pengujian dengan 5.000 event dan 20% duplikat membuktikan sistem tetap responsif dan menghasilkan statistik yang konsisten antara unique_processed dan duplicate_dropped. Seluruh 16 unit test berhasil lulus, mencakup validasi schema, logika deduplication, persistensi, konsistensi endpoint API, dan performa.

Pendekatan at-least-once delivery di sisi publisher dikombinasikan dengan idempotent consumer di sisi aggregator terbukti menjadi solusi yang pragmatis — lebih mudah diimplementasikan dibandingkan exactly-once, namun menghasilkan efek setara di tingkat aplikasi. Keputusan untuk tidak menerapkan total ordering juga tepat karena menghindari overhead koordinasi global yang tidak diperlukan dalam konteks log aggregator.

Sebagai pengembangan lebih lanjut, sistem dapat ditingkatkan dengan menambahkan Bloom filter untuk mempercepat lookup duplikat pada skala sangat besar, serta mengganti SQLite dengan database yang lebih skalabel seperti PostgreSQL apabila diperlukan deployment multi-instance.

Daftar Pustaka

Van Steen, M., & Tanenbaum, A. S. (2023). *Distributed systems* (4th ed.). Maarten van Steen.